

1.Java Lambda Expressions

```
import java.io.*;
import java.util.*;

import java.util.Scanner;

interface performOperation {
    boolean check(int b);
}

public class Solution {
    public performOperation isOdd() {
        return (b) -> b % 2 != 0;
    }

    public performOperation isPrime() {
        return (b) -> {
            if (b < 2)
                return false;
            for (int i = 2; i < Math.sqrt(b); i++) {
                if (b % i == 0) {
                    return false;
                }
            }
            return true;
        };
    }

    public performOperation isPalindrome() {
        return (b) -> {
            int original = b;
            int rev = 0;
            while (b > 0) {
                int digit = b % 10;
                rev = (rev * 10) + digit;
                b = b / 10;
            }
            return original == rev;
        };
    }
}
```

```

    }

    public static void main(String[] args) {
        performOperation op;
        Solution obj = new Solution();
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();
        int i = 0;
        int[] a = new int[t];
        int[] b = new int[t];
        while (i != t) {
            a[i] = sc.nextInt(); // type
            b[i] = sc.nextInt(); // number
            i++;
        }
        for (int j = 0; j < t; j++) {
            if (a[j] == 1) {
                op = obj.isOdd();
                System.out.println(op.check(b[j]) ? "ODD" : "EVEN");
            } else if (a[j] == 2) {
                op = obj.isPrime();
                System.out.println(op.check(b[j]) ? "PRIME" : "COMPOSITE");
            } else if (a[j] == 3) {
                op = obj.isPalindrome();
                System.out.println(op.check(b[j]) ? "PALINDROME" : "NOT
PALINDROME");
            }
        }

        sc.close();
    }
}

```

2.Java Sort

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import java.util.Scanner;
```

```
class Student {
```

```
    protected int id;
```

```
    protected String name;
```

```
    protected double cgpa;
```

```
    public Student(int id, String name, double cgpa) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.cgpa = cgpa;
```

```
    }
```

```
}
```

```
public class Solution1 {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        List<Student> students = new ArrayList<>();
```

```
        int n = Integer.parseInt(scanner.nextLine());
```

```
        for (int i = 0; i < n; i++) {
```

```
            String[] tokens = scanner.nextLine().split("\\s+");
```

```

        int id = Integer.parseInt(tokens[0]);

        String name = tokens[1];

        double cgpa = Double.parseDouble(tokens[2]);

        Student student = new Student(id, name, cgpa);

        students.add(student);

    }

    scanner.close();

    students.stream().sorted((s1, s2) -> {

        if (s1.cgpa - s2.cgpa > 0) {

            return -1;

        } else if (s1.cgpa - s2.cgpa < 0) {

            return 1;

        } else if (s1.name.compareTo(s2.name) < 0) {

            return -1;

        } else if (s1.name.compareTo(s2.name) > 0) {

            return 1;

        } else {

            return Integer.compare(s1.id, s2.id);

        }

    }).map(s -> s.name).forEach(System.out::println);

}

}

```

3.Sorting Comparator

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    // complete this method

    public int compare(Player a, Player b) {

        if(a.score > b.score) {
            return -1;
        } else if (a.score == b.score) {
            return a.name.compareTo(b.name);
        }

        return 1;
    }
}
```

```
    }  
}
```

```
public class Solution {  
  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        int n = scan.nextInt();  
  
        Player[] player = new Player[n];  
        Checker checker = new Checker();  
  
        for(int i = 0; i < n; i++){  
            player[i] = new Player(scan.next(), scan.nextInt());  
        }  
        scan.close();  
  
        Arrays.sort(player, checker);  
        for(int i = 0; i < player.length; i++){  
            System.out.printf("%s %s\n", player[i].name,  
player[i].score);  
        }  
    }  
}
```

